

Task Manager Codebase Exercise

Part 1: Understanding Project Structure

Initial observations before using AI

Looking at the directory structure, my first impression was that this is a straightforward CRUD application built around a single core entity. The presence of a requirements.txt suggested Python. The folder layout appeared to separate concerns loosely, with what looked like a models directory, a services or logic layer, and some kind of CLI entry point. I initially assumed it was a simple command line tool with no database, probably persisting data to a JSON or text file.

My guesses about the technology stack were Python 3, probably no external framework since the requirements file looked minimal, and file based storage.

My guesses about the main components were a Task class or dataclass, some kind of manager or controller handling the operations, and a main entry point handling user input.

After applying the Project Structure prompt

The AI confirmed the general shape of my understanding but corrected one assumption: the project uses an in memory list rather than file persistence by default, which explained why there was no database configuration anywhere. It also pointed out that the project uses enums for task status and priority rather than plain strings, which was something I had missed when skimming. The architecture is closer to a simple repository pattern than I had initially labelled it.

What I had wrong: I assumed persistence to a file. The base implementation stores everything in memory, which means data is lost on exit unless an export or save mechanism is added. This is directly relevant to Part 2.

Key entry point: The main module or equivalent is where execution begins. The TaskManager class is the central component coordinating all operations.

Part 2: Finding Feature Implementation - Task Export to CSV

Initial search approach

I searched for any reference to file, write, open, export, or csv across the codebase. None of these terms appeared in the core implementation files. This told me that no export functionality existed yet, which meant I was not looking for something to extend but something to add from scratch.

I then looked at the Task class to understand what attributes were available for export. The fields included task ID, title, description, due date, priority, status, and creation date.

My hypothesis before the AI prompt

The export feature would most naturally live in a dedicated utility module, something like `export.py` or `file_utils.py`, keeping the TaskManager class clean. The TaskManager would call this utility with a list of tasks, and the utility would handle the `csv.writer` logic. The entry point would then expose an export command.

After applying the Feature Location prompt

The AI agreed with the utility module approach and added a useful observation: Python's built in `csv` module requires no additional dependencies, so `requirements.txt` would not need updating. It also suggested that the method should accept a file path parameter rather than hardcoding a filename, which I had not thought about. It flagged that the Task objects would need to be converted to dictionaries or a list of field values before writing, since `csv.writer` does not handle custom objects directly.

Implementation map:

A new file, `export.py`, would be created. It would contain a single function, `export_tasks_to_csv`, accepting a list of Task objects and a file path string. The TaskManager class would gain one method that retrieves all tasks and passes them to this function. The main entry point would add one new command option for triggering the export.

Part 3: Understanding the Domain Model

Initial entity sketch

From reading the code, I identified three main concepts. Task was the core entity. TaskStatus was an enum with values like `TODO`, `IN PROGRESS`, `DONE`, and possibly `CANCELLED`. TaskPriority was another enum with values like `LOW`, `MEDIUM`, and `HIGH`.

My initial understanding was that a Task is a unit of work with a lifecycle managed through status transitions, and priority is a classification that affects how the task is treated but does not block transitions.

Questions I had going in:

Whether status transitions were enforced or free form. Whether there was any relationship between tasks. Whether due date was a hard constraint or just informational metadata.

After applying the Domain Understanding prompt

The AI confirmed that transitions are not enforced in the base implementation, meaning you can move a task from TODO directly to DONE without passing through IN PROGRESS. This is a deliberate simplicity choice, not an oversight. It also confirmed that tasks are independent with no parent/child or dependency relationships. Due date is purely informational metadata with no automated behaviour attached to it, which sets up Part 4 very well.

Domain glossary:

Task: the primary unit of work tracked by the system. TaskStatus: the lifecycle stage of a task. TaskPriority: a classification indicating urgency or importance. Overdue: a derived concept, not stored as a field, calculated by comparing due date to the current date at runtime. Abandoned: a status value representing a task that was deprioritised and will not be completed.

Part 4: Practical Application

Scenario: Tasks overdue by more than 7 days should be automatically marked as abandoned unless they are high priority.

Files I would modify:

The TaskManager class would receive a new method, auto abandon overdue tasks, containing the core logic. The main entry point would either call this method on startup or expose it as an explicit command. If a scheduled or background approach were needed, that would require discussion with the team first.

Outline of changes:

Inside the new method, I would iterate over all tasks, filter for those whose due date is not None, whose status is not already DONE or ABANDONED, and whose due date is more than 7 days before today. For each task matching this filter, if the priority is not HIGH, the status would be updated to ABANDONED. Tasks with HIGH priority would be skipped and optionally flagged for manual review.

Questions I would raise with the team before implementing:

Whether this logic should run automatically on application startup or only when explicitly triggered. Whether a task with no due date set should be considered overdue or simply excluded. Whether there needs to be a log or audit trail of which tasks were auto abandoned and when. Whether ABANDONED is already an existing status value or needs to be added to the TaskStatus enum.

Final Reflection

Initial vs final understanding

I started thinking this was a simple flat script with file persistence. I finished understanding it as a clean in memory object oriented application with a clear separation between the entity model, the manager layer, and the entry point. The absence of persistence is a design choice, not a gap, and it shapes how features like CSV export need to be thought about.

Most valuable insight from each part

Part 1 taught me not to conflate the presence of a configuration file with the presence of a persistence layer. Part 2 showed me that before designing where a feature goes, you need to understand what data it depends on and whether that data needs transformation. Part 3 clarified that "overdue" is a runtime calculation, not a stored field, which directly shaped how I approached Part 4. Part 4 was where the understanding became concrete and actionable.

Approach to implementing the business rule

I would write the auto abandon method, add a test case covering the HIGH priority exception explicitly, confirm the team's preference on triggering behaviour, and only then merge. The HIGH priority exception is the most likely source of future bugs if not clearly tested.

Strategies for approaching unfamiliar code going forward

Start with the data model before reading any logic. Read the entry point to understand what the system exposes before understanding how it works internally. Form a written hypothesis before using any tool, including AI, because the comparison is where the real learning happens. Do not assume absence of a feature is an oversight; always check whether it is a deliberate simplicity choice before proposing additions.